

# Estatein — WordPress Theme Development

## Development Process Documentation

### 1. Overview

Estatein is a custom WordPress theme built from a Figma UI kit (“Real Estate Business Website UI Template — Dark Theme”). The theme was hand-coded from scratch — no page builder or drag-and-drop plugin — to keep full control over markup, performance, and pixel accuracy against the source design. Three of the six planned pages are complete and dynamic: Home, About Us, and Properties. Property Details, Services, and Contact remain on the roadmap (see Section 5).

### 2. Development Process

#### 2.1 Design-to-code workflow

Rather than estimating spacing and type scale from screenshots, layout values (padding, gaps, corner radius, font size/weight, colors, fills, strokes, shadows) were pulled directly from the Figma file’s REST API export and cross-checked node-by-node against each of the three target frames per page: Desktop (1920px), Laptop (1440px), and Mobile (390px). This caught several details that would have been easy to miss from visual inspection alone — e.g. a background gradient wash behind a hero heading, a drop-shadow on the search bar, and an inline “Read More” link that was part of the card copy rather than a separate element.

A recurring gotcha: a frame’s child order in the JSON does not always match visual top-to-bottom order, so layer order was verified against each element’s absolute Y coordinate before translating structure to markup.

#### 2.2 Design system first

All colors, type sizes, spacing, and radii were centralized into a single `tokens.css` file as CSS custom properties, with a breakpoint override block for the laptop scale. Every other stylesheet references these variables instead of hardcoding values, so the whole site’s scale can be tuned from one place.

#### 2.3 Responsive strategy

Each page ships three explicit breakpoints matching the Figma frames: a desktop base (targeting  $\geq 1920\text{px}$ , with the standard 1596px content container centering itself), a laptop range ( $\leq 1440\text{px}$ ), and a mobile range ( $\leq 480\text{px}$ ). Shared gutter/container logic lives in one `.container` class so every section — header, cards, forms — gets consistent  $162\text{px} \rightarrow 80\text{px} \rightarrow 16\text{px}$  gutters without per-section overrides.

#### 2.4 Componentization & reuse

Structural patterns that repeat across pages (the CTA banner, the footer, the sparkle-accent section intro, the carousel controls, card meta pills) were built once and reused verbatim across Home, About Us, and Properties rather than re-implemented per page.

### 3. Technology & Tooling Choices

- **WordPress core + a fully custom theme** (no Elementor/Divi/etc.) — chosen for clean, inspectable markup and to demonstrate hand-written PHP template and WP\_Query skills rather than page-builder configuration.
- **Advanced Custom Fields (ACF)** — the one required plugin. Used to attach structured metadata (price, bedrooms, rating, role, price tags, etc.) to custom post types, registered as local field groups in `functions.php` (`acf_add_local_field_group`) so field definitions ship with the theme’s code rather than living only in the database.
- **Custom Post Types:** Property, Testimonial, FAQ, Team Member, and Client — each registered with `register_post_type()` and paired with its own ACF field group. This lets all repeatable content (property listings, testimonials, FAQs, team bios, client case studies) be managed from wp-admin instead of hardcoded in templates.
- **Vanilla JavaScript, no libraries** — the mobile nav toggle, announcement-bar dismiss, and a small generic carousel controller are all dependency-free. The carousel reads its items-per-page live from each grid’s CSS grid-template-columns, so one script drives every paginated section (Featured Properties, Testimonials, FAQ, Properties-page categories, Valued Clients) and automatically matches whatever column count the current breakpoint renders — no per-section configuration needed.

- **Native contact form, no form plugin** — the Properties page lead form posts to admin-post.php, is nonce-protected, and sends via wp\_mail() to the site admin address, avoiding a Contact Form 7 / WPForms dependency for a single form.
- **Cache-busted asset loading** — every enqueued stylesheet/script uses filemtime() as its version string, so browsers always pick up the latest CSS/JS during development without manual cache-busting.

## 4. Pages Implemented

### Home

Hero with search-style stats, feature highlights, dynamic Featured Properties (carousel), dynamic Testimonials (carousel), dynamic FAQ (carousel), CTA banner, footer.

### About Us

Our Journey (with stats), Our Values, Our Achievements, a 6-step “Navigating the Estatein Experience” process, Meet the Team (Team Member CPT), Our Valued Clients (Client CPT, carousel), CTA + footer reused from Home.

### Properties

Hero with a search bar and 5 filter dropdowns (Location, Property Type, Pricing Range, Property Size, Build Year), a dynamic “Discover a World of Possibilities” category grid (carousel, reuses the Property CPT with an added tag field), and a full lead-generation form with real server-side handling.

## 5. Known Limitations / Next Steps

- Property Details, Services, and Contact Us pages are not yet built.
- The Properties page search box and 5 filter dropdowns are real form fields but are not yet wired to an actual filtering query — currently visual/structural only.
- Several images (team photos, the “building on hand” journey photo, value icons) are placeholders pending final brand assets from the client.
- No automated test suite; testing so far has been manual, cross-breakpoint visual review against the Figma frames.
- No image optimization/lazy-loading pass yet (planned before final handoff).

## 6. Running the Site Locally

The project targets a standard local PHP/MySQL stack (XAMPP was used during development).

- **1.** Install XAMPP (or any Apache + PHP 8+ + MySQL stack) and start Apache and MySQL.
- **2.** Copy the estatein folder into <xampp>/htdocs/estatein/wp-content/themes/ inside a standard WordPress installation (download the matching WordPress core from wordpress.org if not already present).
- **3.** Create a MySQL database, point wp-config.php at it, and complete the standard 5-minute WordPress install at http://localhost/estatein/.
- **4.** Install and activate the Advanced Custom Fields plugin (required — the theme’s field groups depend on it).
- **5.** Activate the Estatein theme under Appearance → Themes.
- **6.** Under Settings → Permalinks, click Save once to flush rewrite rules for the custom post types.
- **7.** Add sample content: Properties, Testimonials, FAQs, Team Members, and Clients each have their own menu item in wp-admin, with ACF fields shown on the post-edit screen.
- **8.** Create a page titled “About Us” with the “About Us” page template, and a page titled “Properties” (slug: properties) with the “Properties” page template, then publish both.